

PROJECT DOCUMENTATION

Academic Question Answering System Using RAG

Retrieval-Augmented Generation for Source-Cited Academic Answers

Abstract

The Academic Question Answering System uses Retrieval-Augmented Generation (RAG) to answer questions from educational PDF documents. It extracts text, divides the text into smaller passages, creates embeddings, and stores them for semantic retrieval. For each question, relevant passages are supplied to a language model to generate an evidence-grounded response. Source and page citations help readers verify the answer.

Problem Statement

Searching lengthy study PDFs manually can be slow, and basic keyword search may miss relevant passages when the wording differs. General-purpose chatbots may also provide answers without evidence from the intended course material. This project aims to retrieve relevant content from supplied PDFs and produce answers with traceable source references.

Aim

Develop an academic question-answering prototype that uses document evidence and provides verifiable citations.

1. Objectives

- Extract text from PDFs while preserving original page numbers and source filenames.
- Split extracted text into overlapping chunks suitable for semantic retrieval.
- Convert chunks into embeddings and store them in a vector database.
- Retrieve relevant passages for natural-language questions.
- Generate answers grounded in retrieved context and display source/page citations.
- Evaluate retrieval relevance, answer correctness, faithfulness, and citation validity.

2. Scope

The initial prototype is designed for a small collection of approximately 1–5 selectable-text PDFs focused on AI and RAG. The core scope includes extraction, preprocessing, embeddings, vector search, answer generation, citations, and evaluation. Possible future extensions include OCR for scanned documents, additional subjects, conversation history, highlighted citations, and web deployment.

3. Approach and Methodology

Stage 1 — Document ingestion

Collect the PDFs and extract their text. Retain the source filename and original page number so that retrieved passages can be traced back to the document.

Stage 2 — Cleaning and chunking

Remove extraction noise where appropriate and divide the text into manageable, overlapping passages. A starting configuration discussed for this project is about 250 tokens per chunk with 60 tokens of overlap; adjust it based on evaluation.

Stage 3 — Embeddings and indexing

Convert each passage into a numerical embedding and store the vector alongside its text and metadata.

Stage 4 — Retrieval and generation

Embed the user's question, retrieve semantically relevant passages, and pass those passages to the language model as context.

Stage 5 — Citation display

Build citations from stored metadata. The model must not invent filenames or page numbers.

4. System Architecture

The system has two connected workflows: a document-indexing pipeline and a question-answering pipeline.

Document-indexing pipeline

PDF files → text extraction → cleaning and chunking → source/page metadata → embeddings → vector database

Question-answering pipeline

User question → query embedding → similarity search → relevant passages → language model → answer with citations → user interface

Main components

- **PDF processor:** extracts document text and page numbers.
- **Chunking and metadata:** creates passages and preserves source details.
- **Embedding model and vector store:** represents content as vectors and indexes it.
- **Retriever:** selects passages relevant to a question.
- **Generation and citation layer:** creates an evidence-based answer and displays source references.
- **Evaluation module:** tests retrieval, answer quality, faithfulness, and citations.

5. Technology Stack

The implementation may use the following tools, depending on the packages and services actually configured in the project:

- **Python:** core language and pipeline orchestration.
- **PyMuPDF:** PDF text extraction and page-level metadata.
- **Sentence Transformers or the configured embedding model:** semantic representations of text.
- **ChromaDB:** vector storage and similarity retrieval.
- **Google Gemini API:** language-model answer generation, if configured.
- **LangChain and Streamlit:** optional orchestration or interface components, only if used.
- **Git and a virtual environment:** version control and dependency isolation.

Document the exact installed packages, model names, and configuration used in the implementation. Optional tools should not be described as implemented unless they are present in the project.

6. Citation Mechanism and Key Contribution

Each indexed chunk should retain a document identifier, source filename, original page number, chunk identifier, and text. The application should create displayed citations from this metadata rather than asking the language model to invent references. Each citation should be checked to ensure that the cited passage supports the associated claim.

Example citation format: *source_document.pdf — Page 4*. This is only a format example; use actual filenames and page numbers from the indexed dataset.

Key contribution: a document-grounded academic question-answering workflow with traceable, page-level evidence.

- Uses the supplied PDFs as the evidence base.
- Supports semantic matching when a question uses different wording from the source.
- Helps readers locate supporting information in the original document.
- Can indicate when the available evidence is insufficient.
- Allows retrieval, generation, and citation components to be evaluated separately.

7. Feasibility Analysis

Technical: A small prototype is feasible using established Python libraries and selectable-text PDFs. Retrieval relevance and citation validation are key challenges.

Economic: Open-source libraries and local vector storage can reduce costs. Hosted language-model APIs may have usage charges, quotas, or rate limits.

Operational: Users can index documents, ask questions, and inspect citations without needing to understand the underlying algorithms.

Scheduling: Work can be divided into dataset preparation, indexing and retrieval, answer generation, evaluation, and documentation.

8. Requirements

Functional requirements: process PDFs; preserve page metadata; create embeddings; accept questions; retrieve passages; generate answers; display citations; handle unsupported questions; and support evaluation.

Non-functional requirements: answer quality, traceability, reasonable response time, modularity, secure API-key handling, error handling, and reproducible settings.

9. Dataset and Evaluation

The initial dataset consists of a small set of selectable-text PDFs about AI and RAG. For each chunk, retain the source filename, original page, chunk identifier, text, and embedding. Maintain a separate evaluation set containing questions, reference answer points, and expected source pages.

- Direct factual and conceptual questions.
- Paraphrased questions whose wording differs from the source.
- Questions that require evidence from multiple passages.
- Out-of-scope questions that are not supported by the PDFs.
- Citation checks to confirm that source and page references support the answer.

Suggested evaluation measures

- **Retrieval:** Recall@K, Precision@K, and Mean Reciprocal Rank where appropriate.
- **Answer quality:** correctness against reference answers, completeness, and faithfulness to retrieved evidence.
- **Citations:** source validity, evidence support, and page-reference accuracy.

10. Test Plan

- **Covered question:** return a relevant answer and a supporting citation.
- **Paraphrased question:** retrieve the relevant passage despite different wording.
- **Multi-passage question:** combine evidence and identify the sources used.
- **Unsupported question:** state that the available documents do not provide sufficient evidence.
- **Page-reference check:** verify that the cited page exists and supports the answer.
- **Empty input or API failure:** handle the issue without crashing.

Report actual test counts and measured evaluation results. Do not claim a numerical accuracy rate unless it has been measured on the evaluation dataset.

11. Limitations and Future Work

Limitations may include narrow topic coverage, imperfect extraction of tables or equations, missed relevant passages, unsupported language-model claims, dependence on an external API, and limited generalizability from a small evaluation set. Future work could add OCR, hybrid search, reranking, highlighted citations, more documents, conversation history, broader benchmarks, and web deployment.

12. Conclusion

The proposed system demonstrates how document extraction, chunking, embeddings, vector search, and language-model generation can be combined to answer academic questions using retrieved PDF content. Page-level metadata supports citation traceability, while a separate evaluation process helps assess retrieval, answer faithfulness, and citation correctness. Reliability should be established through measured tests on the chosen dataset.